Experiment No. 3

Title: Study of IC-74LS153 Multiplexer&IC 74138 as Decoder.

Problem Definition & Aim of Experiment:

1. Design(truth table) and Implement 8:1 mux using IC 74153.
2. Design (truth table) and Implement Boolean function (Any function can be given in exam.)
3. Design (truth table) Full adder implementation using hardware reduction table.
4. Design (truth table) Full Subtractor implementation using hardware reduction table.
5. Design (truth table)and Full Adder & Subtractor using IC 74138.

Objective of Experiment:

To understand how multiplexer can be used to reduce the hardware required.

Lab Equipment'sand IC's Used:

Digital Trainers Kit74LS153,74LS138, 74LS32, 74LS04.

AIM: Part A – MUX IC 74153

1) Verification of IC.
2) Implementation of 8:1 Mux by cascading 2, 4:1 mux in IC 74153

'Part B – Decoder IC 74138

1) Verification of IC.
2) Boolean function implementation.

THEORY :

Digital Multiplexer:

Multiplexer are combinational digital circuits equating as controlled switches with several data inputs (I_0, I_1, I_2 ...) & one single data output ("out").At any time one of the I/p is transmitted to output. According to binary signals applied on control pairs to circuit. Usually the number of data inputs is a power of two. Multiplexing is the process of transmitting a large no. of information units over a small no. of channel / digital multiplexer is a combinational large circuit which performs the operation of multiplexing .It selects the operation of multiplexing. It selects the operation of binary information from one of the many input lines & transfer to a single o/p line. Multiplexer is called a data selector or multiposition switch because it selects one of the many input. Selection of a particular line is

controlled by a set of a selection lines or selects inputs. The number of select lines depends upon no. of input lines.

Generally there is 'n' selects line for 'm' input lines. By applying a particular code on select lines is transmitted on the output lines.Block diagram of MUX is shown. at contains '2^m' input lines 'm' select & one unable input which is used to activate or dedicate MUX . Depending upon the no. of I/P & O/P lines various types of multiplexers are available. We have 2:1, 4:1, 8:1, 16:1 MUX. Here the first no. indicates the no. of input lines & second no. indicates the no. of output lines.

Demultiplexer :

Demultiplexer is a logic used to perform exactly reverse function performed by multiplexer. It accepts a single input and distributes among several outputs. The selection of a particular output line is controlled by a set of selection line. There are n input lines & 2^m is the number of selection line whose bit combinations determine which output to be selected.

Difference between Multiplexer, Demultiplexer & Decoder

Point	Multiplexer	Demultiplexer	Decoder
Input	Many input lines	Single input line	Many input line also Acts as select line
Output	Single output line	Many output lines	Many output line, Active low output
Select line	$2^m = n$	$n = 2^m$	Enable inputs used

Encoder & Decoder :

1. Encoders are used to encode given digital number into different numbering format like decimal to BCD Encoder, Octal to Binary.
2. Decoders are used to decode a coded binary word like BCD to seven segment decoder.
3. Thus encoder and decoder are application specific logic develop, we can not use any type of input for any encoder and decoder.

4. Need to select input according to encoder and decoder being selected for a particular application as mention in examples above.

Uses of Mux. :

1) Use for Boolean function implementation.
2) Construct a common bus system.
   1) To select between multiple sources & signal destination.
   2) Inter register transfer.

Advantages :

1) Simplification of logic expression not required.
2) Logic design is simplified.

Disadvantage :

Only one function can be implemented using one MUX. Hence they can't be used in combinational logic circuit which contains many function.

Part-A (IC 74153)

1. VERIFICATION OF IC 74153 :

IC 74153 is a dual layer 4:1 MUX. It has four input lines for (I_0D-I_3D) for second MUX & active high output. 'Y_a', 'Y_b' (1Y or 2Y). It has select lines S_1S_0 common to both MUX. The Enable inputs are active low, E_a& E_b (1G and 2G). The MUX is activated when they are at logic o.

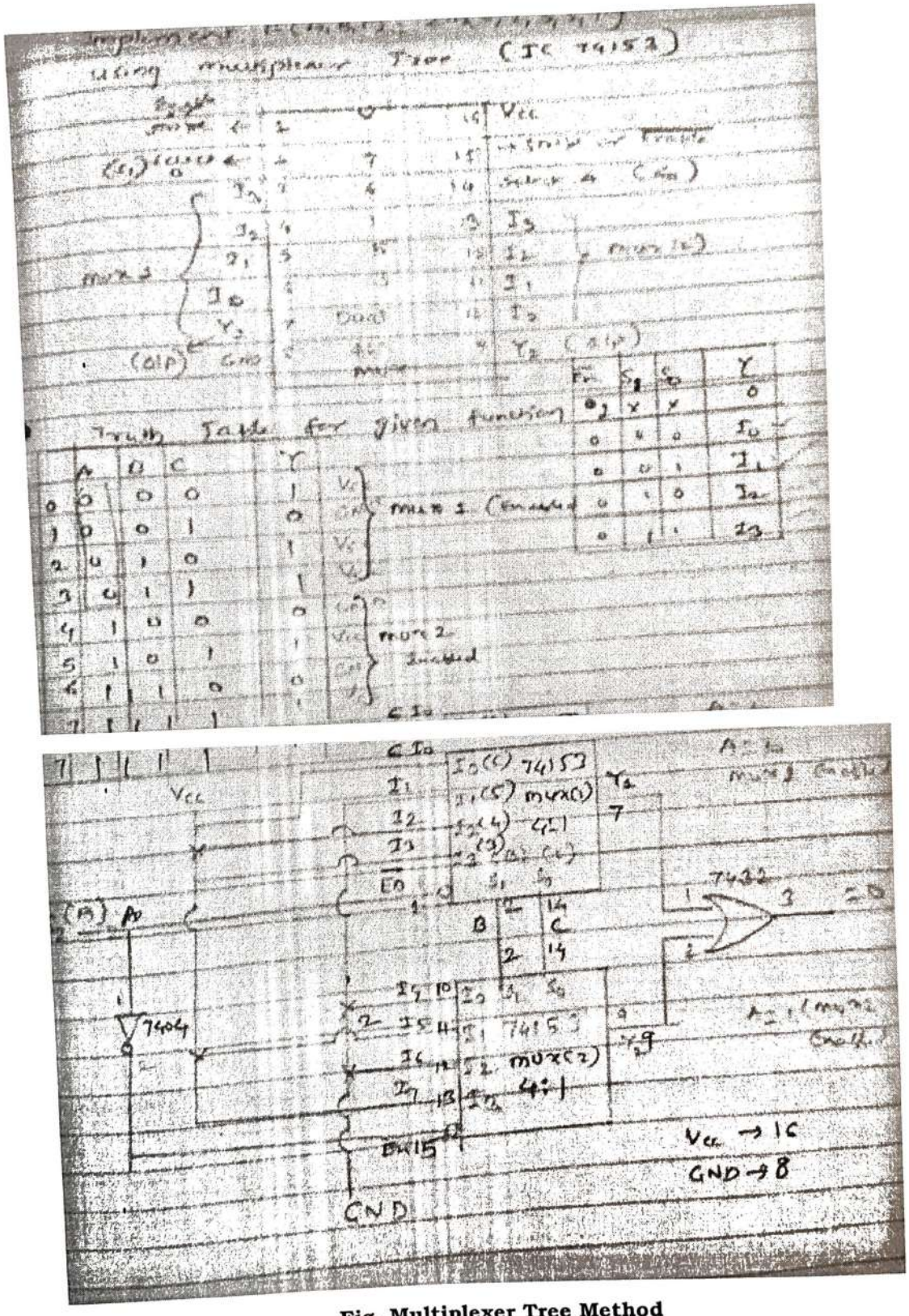


Fig. Multiplexer Tree Method

FUNCTION IMPLEMENTATION:

$Y = \sum m (1, 3, 5, 6)$

Thisexpression is in Standard SOP form and it is three variable function. So, we need to use mux with three select inputs i.e. 8:1 Mux. Already we have implemented 8:1 Mux using IC 74153. For Boolean function in Standard SOP form we connect data inputs corresponding to the minterms present in the given function to Vcc and remaining data inputs to ground.

Truth table :

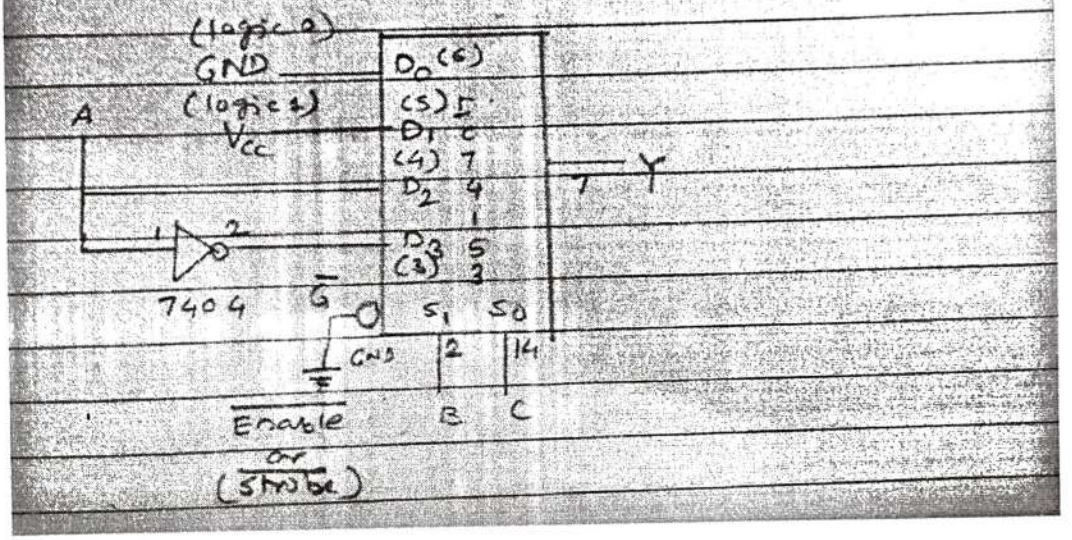
Inputs			*Output*
C	***B***	***A***	***Y***
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	0

$Y = \Sigma m(1, 3, 5, 6)$

Using Hardware Reduction Table (MSA) Approach

	D_0	D_1	D_2	D_3
$\bar{A}$	0	(1)	2	(3)
A	4	(5)	(6)	7
	0	1	A	$\bar{A}$

Fig. Hardware Reduction Table



4. IMPLEMENTATION OF FULL ADDER USING IC 74153:

A full adder is a combinational circuit that forms the arithmetic sum of three input bits. It consists of three inputs and two outputs. Two of these variables denoted by A and B represent the two significant bits to be added. The third input represents the carry from previous lower significant position.

Truth Table for Design of full adder:

Input			***Output***	
A	B	C	Sum	Carry
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

Sum= $\sum$m (1, 2, 4, 7),

Carry=$\sum$m (3, 5, 6, 7)

Equations for Full Adder

Sum & Carry

Sum = $\Sigma m(1,2,4,7) = F(A,B,C)$

Carry = $F(A,B,C) = \Sigma m(3,5,6,7)$

Sum:-

	D_0	D_1	D_2	D_3
$\bar{A}$	0	(1)	(2)	3
A	(4)	5	6	(7)
	A	$\bar{A}$	$\bar{A}$	A

Carry:-

	D_0	D_1	D_2	D_3
$\bar{A}$	0	1	2	(3)
A	4	(5)	(6)	(7)
	0	A	A	1

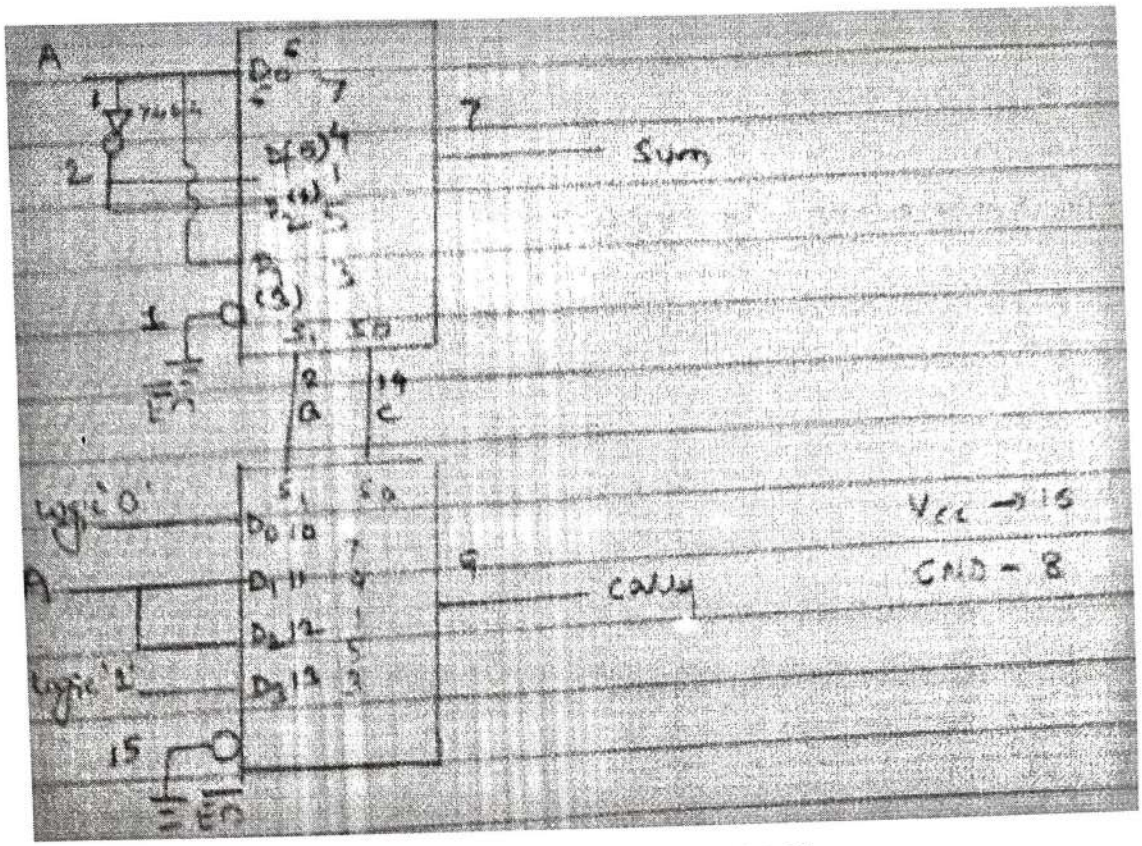


Fig. Circuit Diagram for Full Adder

Hardware Requirements :

GATE	Quantity	IC	Quantity
Mux.	1	74153	1
NOT	1	7404	1

Part-B Decoder (IC 74138)

THEORY:

Discrete quantities of information are requested in digital system with binary codes. A binary code of n bits is capable of representing into 2^n distinct elements of the coded information.

Decoder converts coded input to coded outputs accepts one of the code.

There are different types of decoders such as 3:8 decoder, 4:16 line decoders etc. These are in general called as n: m line decoder where $m=2^n$ and n= no. of input lines and m=no. of output lines.

Demux also takes one input data line source and selectively distributes it to one of n output channels. The only difference between demux and decoder is that demux has Din (data i/p) line whereas decoder does not have.

ADVANTAGES:

1) The decoder provides best implementation whenever there are many outputs of the combinational circuit and each o/p of the function (or its complement) is required to be expressed with a small no. of minterms.
2) The decoder can function as demux. If the Enable i/p line is taken as Din (data i/p) .

DISADVANTAGES:

Since decoder method requires an OR gate for each o/p function, so there is new hardware used. And it is always advisable to use minimum hardware as we come across problems like propagation delay of gates.

APPLICATIONS:

Decoder is worthily used for decoding binary information and memory interfacing. It is used for the implementation of Boolean function.

A) Verification of IC 74138:

We use IC 74138 which accepts 3 binary weighted inputs (A0, A1, A2) and when enabled provides mutually exclusive active low outputs (y0-y7). It features 3 Enable i/ps. Two active low (G2A, G2B) and one active high (G1). Every output will be high unless G2A, G2B are low and G1 is high. It has demultiplexing capability and multiple enable i/ps for easy expansion.

Function Table of 3:8 decoder:

Input						Output							
Enable			Data										
G2A	G2B	G1	A2	A1	A0	Y0	Y1	Y2	Y3	Y4	Y5	Y6	Y7
0	0	0	X	X	X	1	1	1	1	1	1	1	1
0	1	1	X	X	X	1	1	1	1	1	1	1	1
1	0	1	X	X	X	1	1	1	1	1	1	1	1
1	1	1	X	X	X	1	1	1	1	1	1	1	1
0	0	1	0	0	0	0	1	1	1	1	1	1	1
0	0	1	0	0	1	1	0	1	1	1	1	1	1
0	0	1	0	1	0	1	1	0	1	1	1	1	1
0	0	1	0	1	1	1	1	1	0	1	1	1	1
0	0	1	1	0	0	1	1	1	1	0	1	1	1
0	0	1	1	0	1	1	1	1	1	1	0	1	1
0	0	1	1	1	0	1	1	1	1	1	1	0	1
0	0	1	1	1	1	1	1	1	1	1	1	1	0

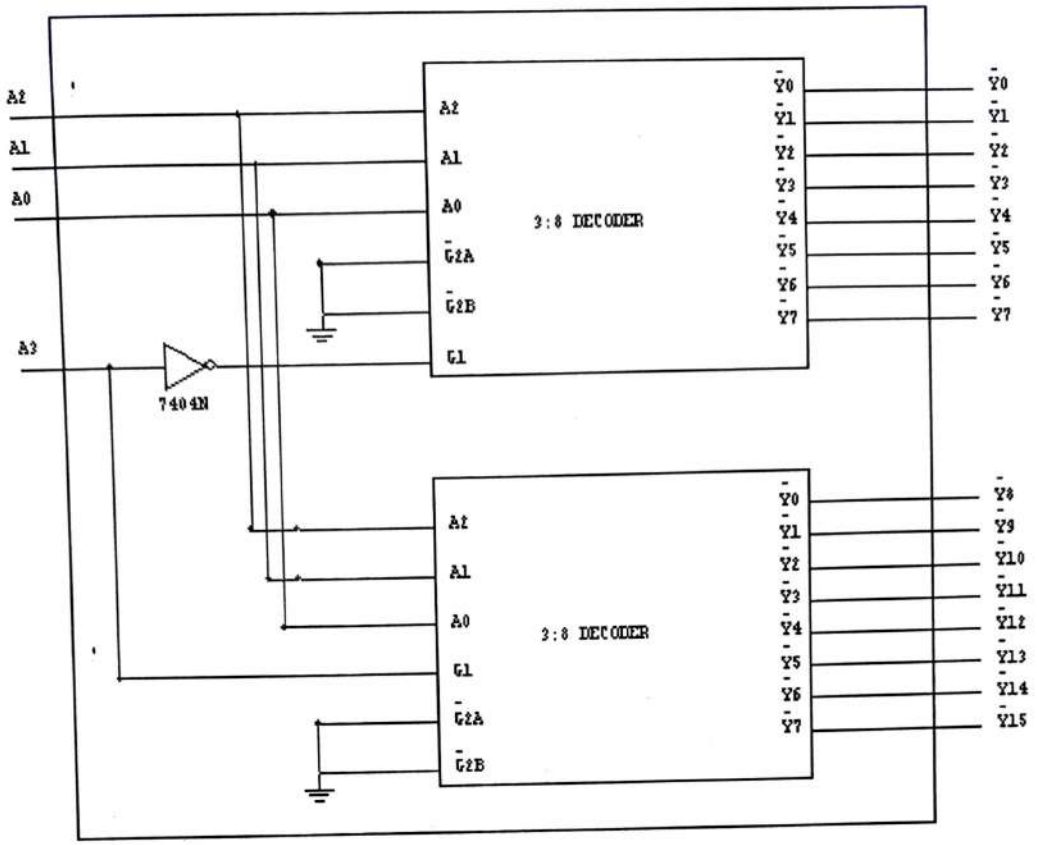
B) Cascading of IC 74138:

The enable i/p G1 active high of IC 74138 is used for cascading. for cascading 2 IC's ,the enable i/p G1 of first IC is connected to G1 enable i/p of second IC through a NOT gate. This enable i/p is used as MSB select i/p line A3. the other three select input lines of both IC's (A0,A1,A2) are also shorted to select input lines of second IC to get single i/p select lines (A0,A1,A2).

The i/p line A3 is used to enable /disable the 2 IC 74138 decoders. When A3=0, first IC is enabled and second is disabled. Thus the first decoder will generate minterms from 0000 to 0111 as o/p and the second decoder will generate nothing. When A3=1, the enable conditions are reversed and thus second decoder IC will generate minterms 1000 to 1111.

Function Table of 4:16 decoder using IC 74138 (3:8 decoder):

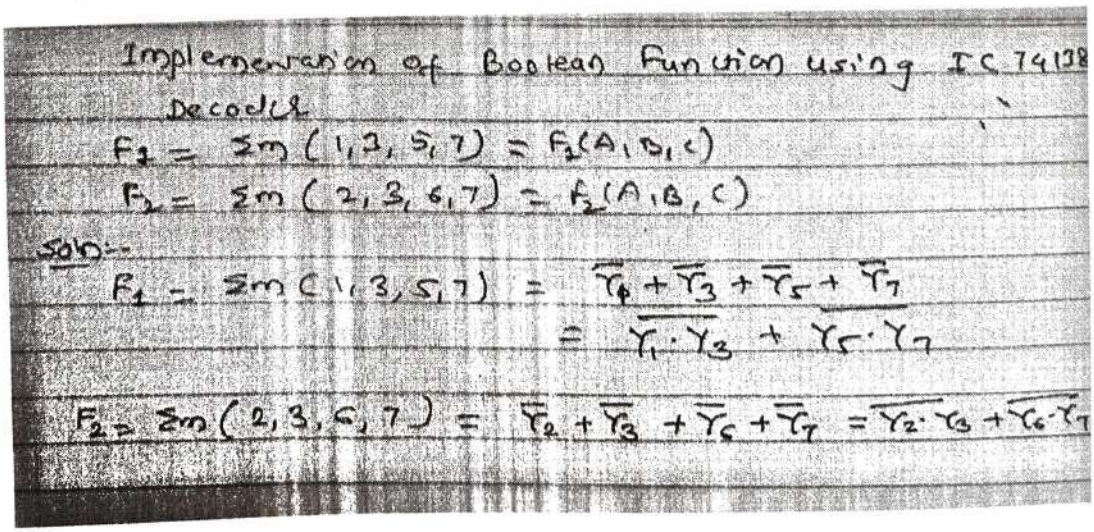
Input						Output															
Enable		Data																			
G2A	G2B	A3	A2	A1	A0	Y0	Y1	Y2	Y3	Y4	Y5	Y6	Y7	Y8	Y9	Y10	Y11	Y12	Y13	Y14	Y15
0	1	X	X	X	X	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1
1	0	X	X	X	X	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1
1	1	X	X	X	X	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1
0	0	0	0	0	0	0	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1
0	0	0	0	0	1	1	0	1	1	1	1	1	1	1	1	1	1	1	1	1	1
0	0	0	0	1	0	1	1	0	1	1	1	1	1	1	1	1	1	1	1	1	1
0	0	0	0	1	1	1	1	1	0	1	1	1	1	1	1	1	1	1	1	1	1
0	0	0	1	0	0	1	1	1	1	0	1	1	1	1	1	1	1	1	1	1	1
0	0	0	1	0	1	1	1	1	1	1	0	1	1	1	1	1	1	1	1	1	1
0	0	0	1	1	0	1	1	1	1	1	1	0	1	1	1	1	1	1	1	1	1
0	0	0	1	1	1	1	1	1	1	1	1	1	0	1	1	1	1	1	1	1	1
0	0	1	0	0	0	1	1	1	1	1	1	1	1	0	1	1	1	1	1	1	1
0	0	1	0	0	1	1	1	1	1	1	1	1	1	1	0	1	1	1	1	1	1
0	0	1	0	1	0	1	1	1	1	1	1	1	1	1	1	0	1	1	1	1	1
0	0	1	0	1	1	1	1	1	1	1	1	1	1	1	1	1	0	1	1	1	1
0	0	1	1	0	0	1	1	1	1	1	1	1	1	1	1	1	1	0	1	1	1
0	0	1	1	0	1	1	1	1	1	1	1	1	1	1	1	1	1	1	0	1	1
0	0	1	1	1	0	1	1	1	1	1	1	1	1	1	1	1	1	1	1	0	1
0	0	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	0



C) Implementation of Boolean function:

The procedure for implementation of combinational circuit by means of a decoder and 'OR' gates requires that the Boolean function fir the circuit be expressed in Sum of Minterms. These forms can be obtained by expanding the function. A decoder is then chosen which generates all the minterms of n i/p variables. The i/p to each OR gate are selected from the decoder outputs according to the minterms list in each function.

For example, **F1=∑m (1, 3, 5, 7) and F2=∑m (2, 3, 6, 7)**



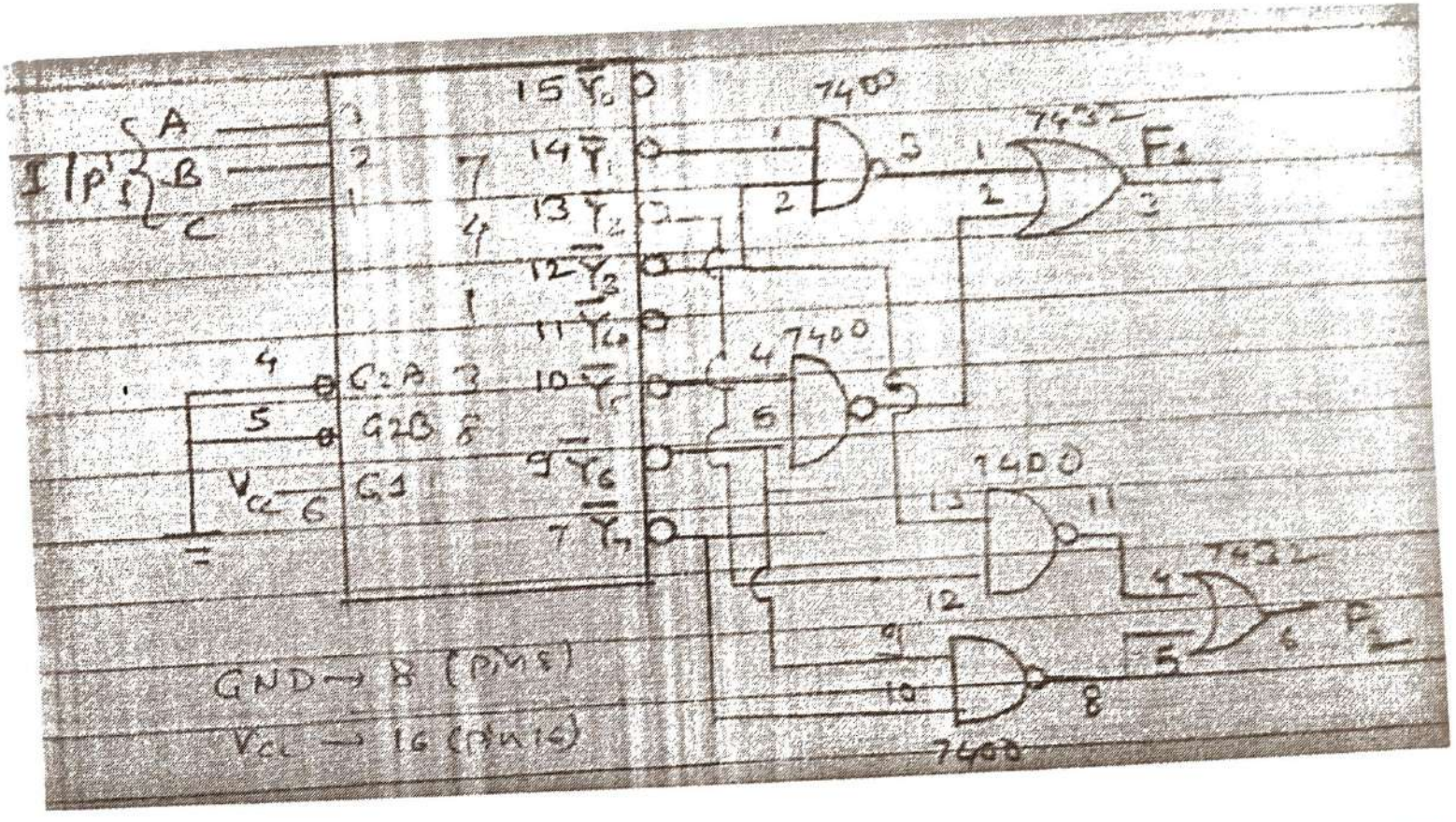
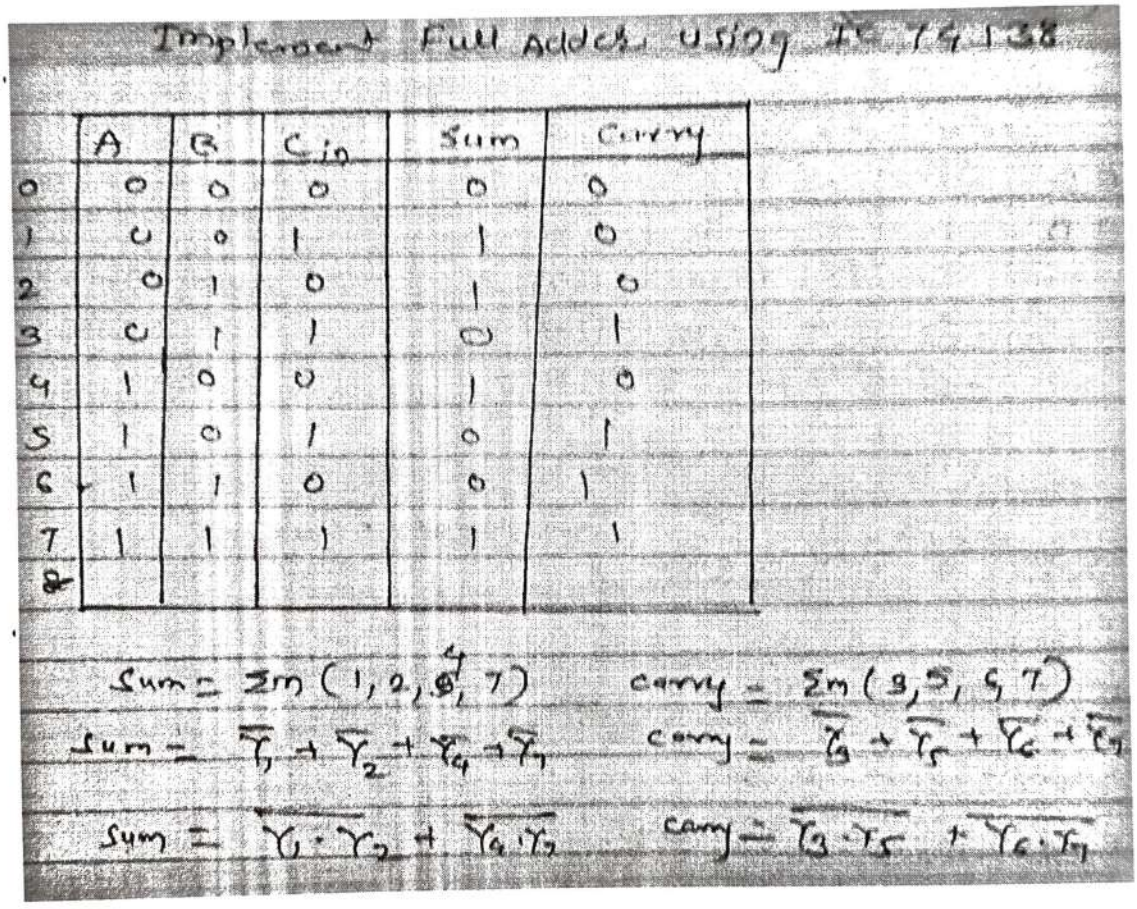


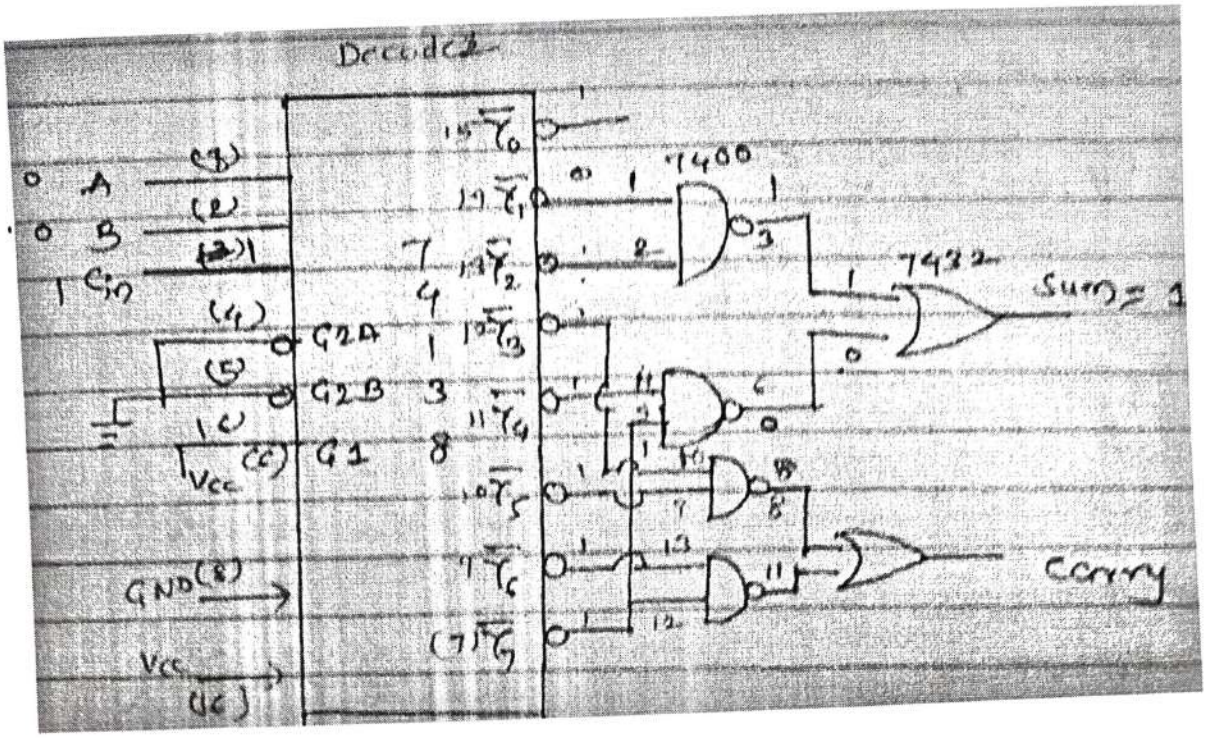
Fig. Logic Circuit diagram for Function Implementation using IC 74138 Decoder

Implementation of Full Adder using IC 74138:-

First of all we need to decide on which type of decoder the above Boolean function can beimplemented. The highest minterm is 7 and minimum no. of bits required to represent it in binary form are 3. So we have 3 select lines in 3:8 decoders so we can use IC 74138.

To implement the function we require two OR gates and four NAND gates (7432& 7400). As the o/p of the decoder IC 74138 are active low and we need to get o/p active high at the o/p pin of the function SUM and CARRY when respective minterms are selected.





Hardware Required:-

Sr.No.	IC Number	Quantity	Name of IC
1.	74LS138	1	3:8 Decoder
2.	74LS00	1 (4 gates used)	Quad 2 input & 1 output NAND gate
3.	74LS32	1(2 gates used)	Quad 2 input & 1 output NAND gate
4.	Patch chords	Min 26 required	

Implementation of Full Subtractor:

Same case will happen in this case. Again first of all we need to decide on which type of decoder the above Boolean function can be implemented. The highest minterm is 7 and minimum no. of bits required to represent it in binary form are 3. So we have 3 select lines in 3:8 decoders so we can use IC 74138.

To implement the function we require 2-OR gates and 3-NAND gates (7432 & 7400). As the o/p of the decoder IC 74138 are active low and we need to get o/p active high at the o/p pin of the function DIFFERENCE and BORROW when respective minterms are selected.

Truth Table for design of Full Subtractor:

INPUT			OUTPUT	
A2	A1	A0	DIFFERENCE	BORROW
A	B	C		
0	0	0	0	0
0	0	1	1	1
0	1	0	1	1
0	1	1	0	1
1	0	0	1	0
1	0	1	0	0
1	1	0	0	0
1	1	1	1	1

Difference $= \Sigma m(1, 2, 4, 7) = \bar{Y}_1 + \bar{Y}_2 + \bar{Y}_4 + \bar{Y}_7$

$= \overline{Y_1 \cdot Y_2} + \overline{Y_4 \cdot Y_7}$

Borrow $= \Sigma m(1, 2, 3, 7) = \bar{Y}_1 + \bar{Y}_2 + \bar{Y}_3 + \bar{Y}_7$

$= \overline{Y_1 \cdot Y_2} + \overline{Y_3 \cdot Y_7}$

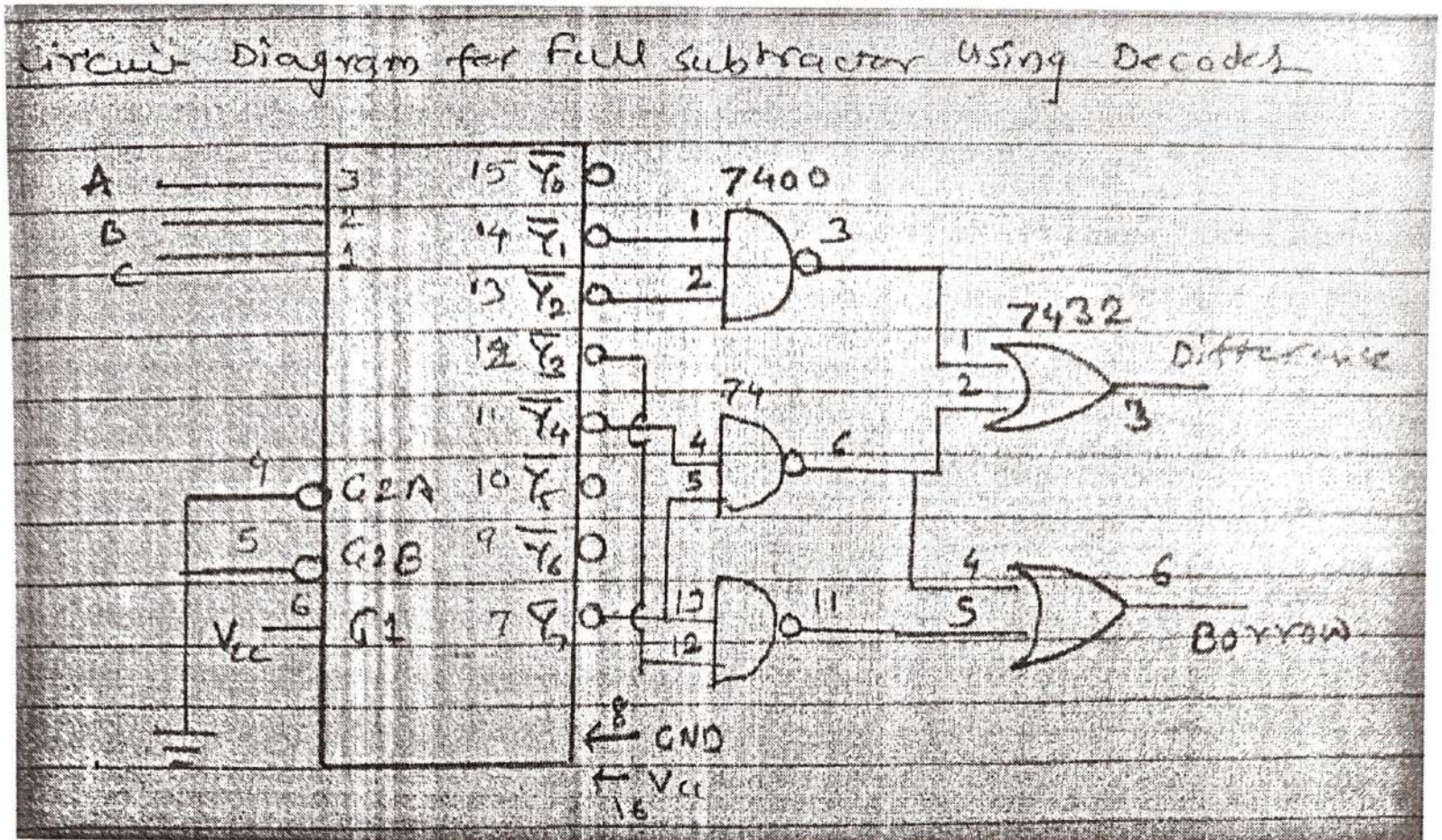


Fig. Circuit diagram for Full Subtractor Decoder using IC 74LS138

Experiment No. 4

Title:- Design & Implement Asynchronous & Synchronous Counter

AIM: To design and implement 3 bit UP, Down Ripple & Synchronous Counter using MS-JK Flip-flop.

OBJECTIVE:

- To understand concept and role of clock in sequential circuits.
- To understand the design procedure of asynchronous & Synchronous counter.

ICs USED : IC 7476 (MS-JK Flip-flop), IC 7408(Quad 2 i/p AND Gate),

THEORY:

Counters : counters are logical device or registers capable of counting the no. of states or no. of clock pulses arriving at its clock input where clock is a timing parameter arriving at regular intervals of time, so counters can be also used to measure time & frequencies. They are made up of flip flops. Where the pulse are counted to be made of it goes up step by step & the o/p of counter in the flip flop is decoded to read the count to its starting step after counting n pulse in case of module counters.

Types of Counters:

Counter are of two types:

1) Asynchronous counter.

2) Synchronous counter.

Asynchronous counter:

A digital counter is a set of flip flop. The flip flop are connected such that their combined state at any time is binary equivalent of total no. of pulses that have occurred up to that time. Thus its name implies a counter is used to count pulse. A counter is used as frequency dividers. To obtain waveform with frequency that is specific fraction of clock frequency.

Counter may be Asynchronous or synchronous. The Asynchronous counter is also called as ripple counter .An Asynchronous counter uses T flip flop to perform a counting function. The actual hardware used is usually J-K flip flop with J & K connected to logic1. Even D flip flops may be used here.

In asynchronous counter commonly called ripple counter, the first flip-flop is clocked by the external clock pulse & then each successive flip-flop is clocked by the Q or Q' output of the previous flip-flop. Therefore in an asynchronous counter

the flip-flop's are not clocked simultaneously. The input of MS-JK is connected to VCC because when both inputs are one output is toggled. As MS-JK is negative edge triggered at each high to low transition the next flip-flop is triggered.

Synchronous Counter :

When counter is clocked such that each flip flop in the counter is triggered at the same time, the counter is called as synchronous counter. The gates propagation delay at reset time will not be present or we may say will not occur.

1) Asynchronous Up Counter:

Fig. 1 shows 3bit Asynchronous Up Counter. Here Flip-flop 2 act as a MSB Flip-flop and Flip-flop 0 act as a LSB Flip-flop. Clock pulse is connected to the Clock of Flip-flop 0. Output of Flip-flop 0(Q_0) is connected to clock of next flip-flop (i.e Flip-flop 1) and so on. As soon as clock pulse changes output is going to change (at the negative edge of clock pulse) as a Up count sequence. For 3 bit Up counter state table is as shown below.

State Table :

Counter States	Count		
	Q_2	Q_1	Q_0
0	0	0	0
1	0	0	1
2	0	1	0
3	0	1	1
4	1	0	0
5	1	0	1
6	1	1	0
7	1	1	1
8	0	0	0

Logic diagram :

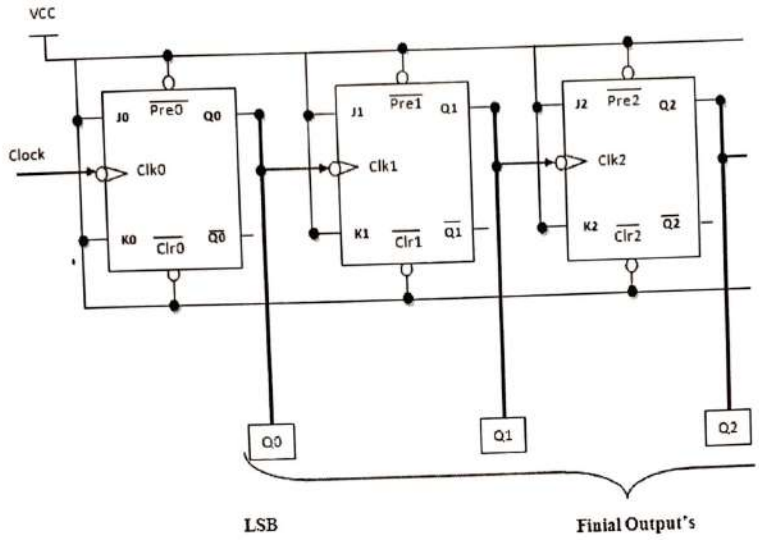


Fig 1: 3 Bit Asynchronous Up Counter

Hardware requirements :

Gate / Flip flop	Quantity	IC	Quantity
MS JK	3	7476	2

2) Down Counter:

Fig. 2 shows 2 bit Asynchronous Down Counter. Here Flip-flop 2 act as a MSB Flip-flop and Flip-flop 0 act as a LSB Flip-flop. Clock pulse is connected to the Clock of Flip-flop 0. Output of Flip-flop 0 (Q_0') is connected to clock of next flip-flop (i.e Flip-flop 1) and so on. As soon as clock pulse changes output is going to change (at the negative edge of clock pulse) as a down count sequence. For 3 bit down counter sate table is as shown below.

In both the counters Inputs J and K are connected to Vcc, hence J-K Flip flop work in toggle mode. Preset and Clear both are connected to logic 1.

State Table :

Counter States	Count		
	Q_2	Q_1	Q_0
7	1	1	1
6	1	1	0
5	1	0	1
4	1	0	0
3	0	1	1

Logic diagram :

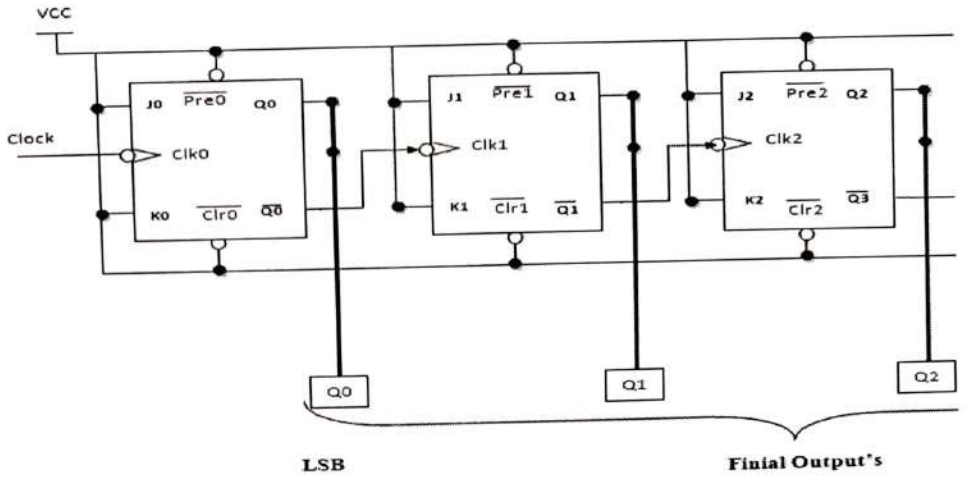


Fig 2: 3 Bit Asynchronous Down Counter

Hardware requirements :

Gate / Flip flop	Quantity	IC	Quantity
MS JK	3	7476	2

Applications :

The asynchronous counters are specially used as the counting devices.

They are also used to count number of pulses applied.

It also works as frequency divider.

It helps in counting the number of product coming out of the machinery where product is coming out at equal interval of time.

Types of synchronous counter:

1) Up counter.

2) Down counter.

1. A 3 bit Synchronous up counter:

The up counter counts from 0 to7 i.e.(000 to 111).for this we are using MS JK flip flop. In IC 74LS76, 2 MS J-K flip flops are present. The clock pulse is given at pin 1 & 6 of the 1st IC & pin 1 of 2nd IC. Next state decoder logic is designed with the help of state table.

State table for synchronous up counter:

Present state			Next state			Flip flop 3		Flip flop 2		flip flop 1	
Q2	Q1	Q0	Q2	Q2	Q0	J2	K2	J1	K1	J0	K0
0	0	0	0	0	1	0	x	0	X	1	x
0	0	1	0	1	0	0	x	1	X	x	1
0	1	0	0	1	1	0	x	x	0	1	x
0	1	1	1	0	0	1	x	x	1	x	1
1	0	0	1	0	1	x	0	0	X	1	X
1	0	1	1	1	0	x	0	1	X	x	1
1	1	0	1	1	1	x	0	x	0	1	X
1	1	1	0	0	0	x	1	x	1	x	1

K-Map :

Q2 \ Q1Q0	00	01	11	10
0	0	0	1	0
1	X	X	X	X

$J_2 = Q_1Q_0$

Q2 \ Q1Q0	00	01	11	10
0	X	X	X	X
1	0	0	1	0

$K_2 = Q_1Q_0$

Q2 \ Q1Q0	00	01	11	10
0	0	1	X	X
1	0	1	X	X

$J_1 = Q_0$

Q2 \ Q1Q0	00	01	11	10
0	X	X	1	0
1	X	X	1	0

$K_1 = Q_0$

Q2 \ Q1Q0	00	01	11	10
0	1	X	X	1
1	1	X	X	1

$J_0 = 1$

Q2 \ Q1Q0	00	01	11	10
0	X	1	1	X
1	X	1	1	X

$K_0 = 1$

Logic Diagram:

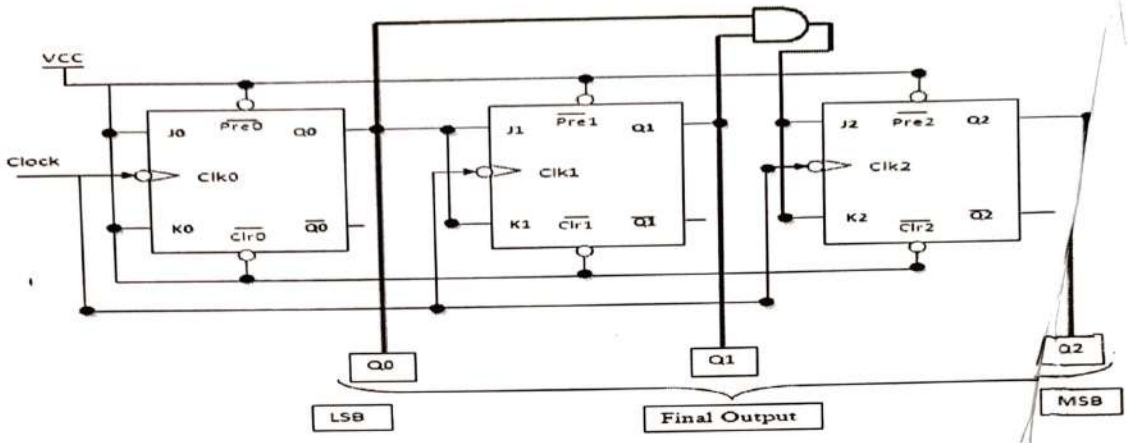


Fig 1: 3 bit Synchronous up counter

2. A 3 bit Synchronous down counter:

This is used to count from 7-0 i.e.(111-000).for this also 2 IC's of 74LS76 are required & hence we use 3 MS JK flip flops. Here also clock is given to 1st& 6th pin of 1st IC & 1st pin of 2nd IC enabling to apply clock to all flip flop at a time. Next state decoder logic is designed with the help of state table.

'State table for synchronous down counter :

Present state			Next state			Flip flop 3		Flip flop 2		Flip flop 1	
Q2	Q1	Q0	Q2	Q1	Q0	J2	K2	J1	K1	J0	K0
1	1	1	1	1	0	X	0	X	0	X	1
1	1	0	1	0	1	X	0	X	1	1	X
1	0	1	1	0	0	X	0	0	X	X	1
1	0	0	0	1	1	X	1	1	X	1	X
0	1	1	0	1	0	0	X	X	0	X	1
0	1	0	0	0	1	0	X	X	1	1	X
0	0	1	0	0	0	0	X	0	X	X	1
0	0	0	1	1	1	1	X	1	X	1	X

$K_0 = 1$

Logic Diagram:

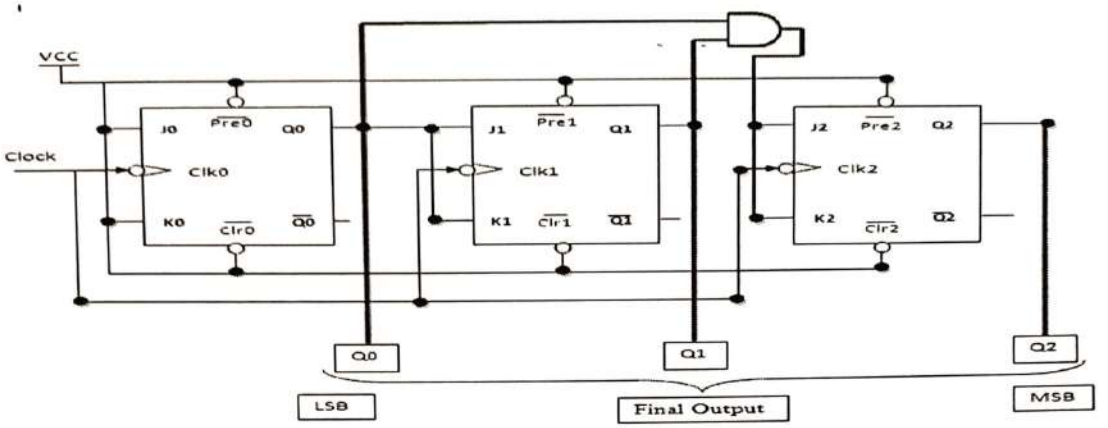


Fig 1: 3 bit Synchronous up counter

2. 'A 3 bit Synchronous down counter:

This is used to count from 7-0 i.e.(111-000).for this also 2 IC's of 74LS76 are required & hence we use 3 MS JK flip flops. Here also clock is given to 1st& 6th pin of 1st IC & 1st pin of 2nd IC enabling to apply clock to all flip flop at a time. Next state decoder logic is designed with the help of state table.

State table for synchronous down counter :

Present state			Next state			Flip flop 3		Flip flop 2		Flip flop 1	
Q2	Q1	Q0	Q2	Q1	Q0	J2	K2	J1	K1	J0	K0
1	1	1	1	1	0	X	0	X	0	X	1
1	1	0	1	0	1	X	0	X	1	1	X
1	0	1	1	0	0	X	0	0	X	X	1
1	0	0	0	1	1	X	1	1	X	1	X
0	1	1	0	1	0	0	X	X	0	X	1
0	1	0	0	0	1	0	X	X	1	1	X
0	0	1	0	0	0	0	X	0	X	X	1
0	0	0	1	1	1	1	X	1	X	1	X

K-Map :

Q1Q0 / Q2	00	01	11	10
0	1	0	0	0
1	X	X	X	X

$J_2 = Q'_1 Q'_0$

Q1Q0 / Q2	00	01	11	10
0	X	X	X	X
1	1	0	0	0

$K_2 = Q'_1 Q'_0$

Q1Q0 / Q2	00	01	11	10
0	1	0	X	X
1	1	0	X	X

$J_1 = Q_0'$

Q1Q0 / Q2	00	01	11	10
0	X	X	0	1
1	X	X	0	1

$K_1 = Q_0'$

Q1Q0 / Q2	00	01	11	10
0	1	X	X	1
1	1	X	X	1

$J_0 = 1$

Q1Q0 / Q2	00	01	11	10
0	X	1	1	X
1	X	1	1	X

$K_0 = 1$

Logic Diagram :

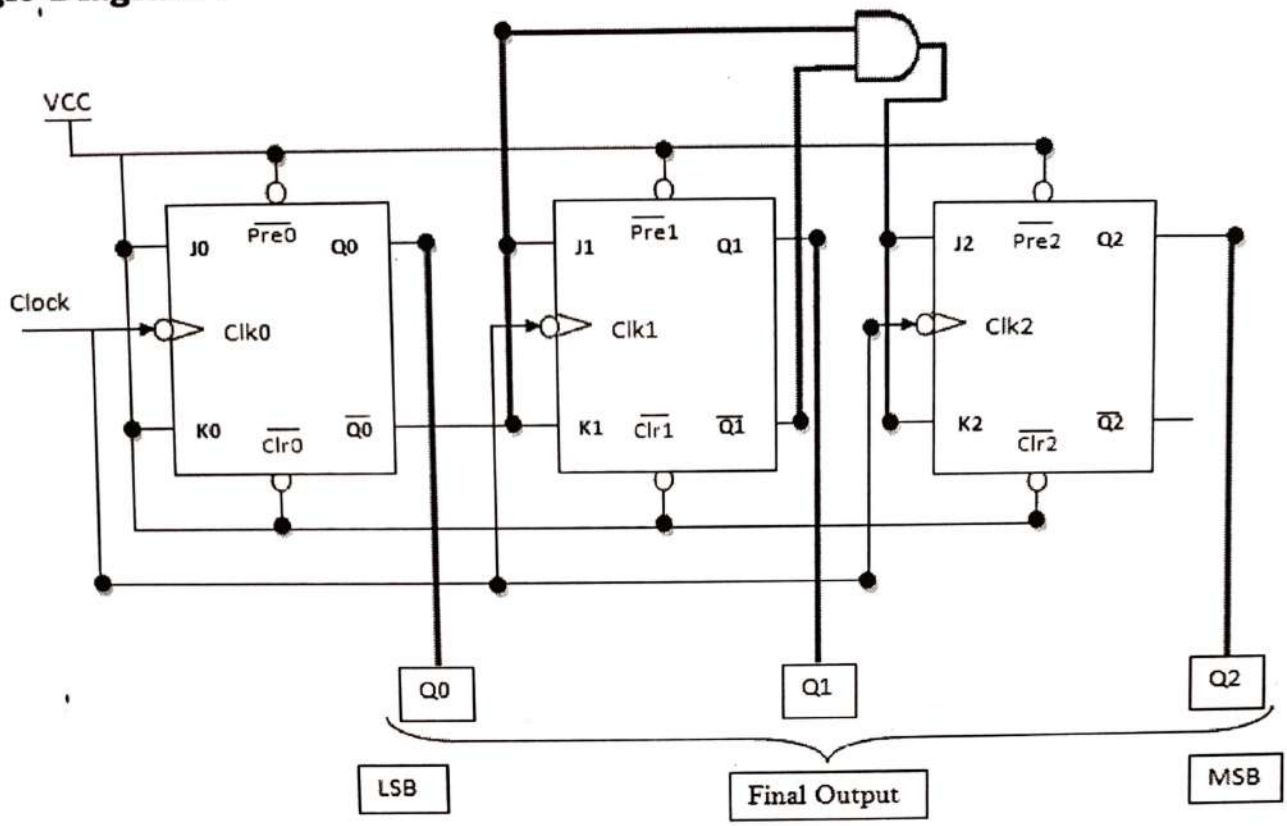


Fig : A 3 bit Synchronous down counter

Uses:

1. Specially used as the counting devices.
2. Used in frequency divider circuit.
3. Used in digital voltmeter.
4. Used in counter type A to D converter.
5. Used for time measurement..
6. It helps in counting the no of product coming out from machinery where product is coming out at equal interval of time.

Conclusion:

Up and down counters are successfully implemented, the counters are studied & o/p are checked. The state table is verified.

Experiment No. 5

Title:- Modulus n Counter

AIM: To design and implement mod - 10, mod – 7, mod - 99 asynchronous BCD counter using IC 7490.

OBJECTIVE: To know difference between regular & truncated counter as well as binary & BCD Counter

IC's USED: IC 7490, Basic gates 7408,7432.

THEORY: IC 7490

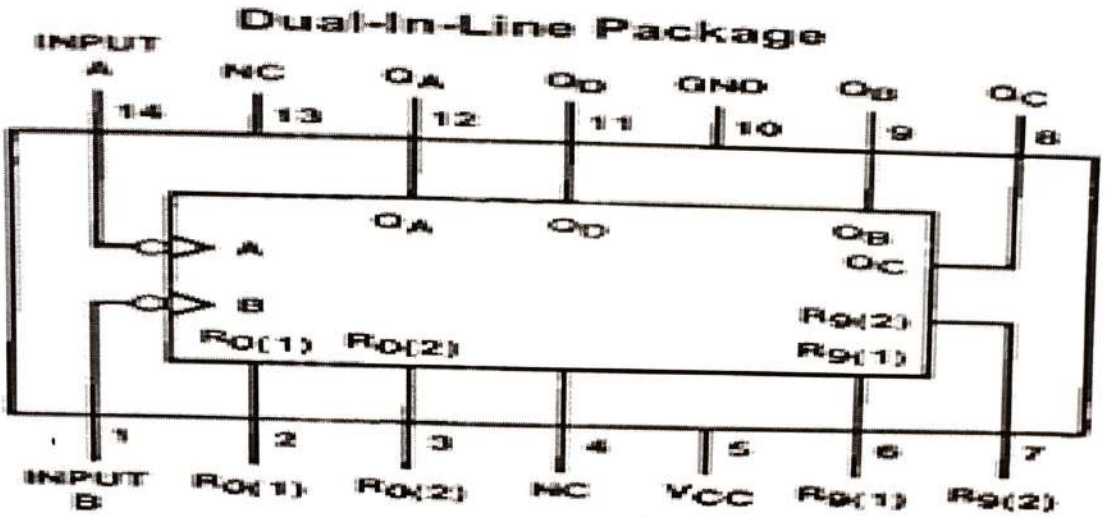
IC 7490 is a TTL MSI (medium scale integration) decade counter. It contains 4 master slave flip flops internally connected to provide MOD-2 i.e. divide by 2 and MOD-5 i.e. divide by 5 counters. MOD-2 and Mod-5 counters can be used independently or in cascading.

It is a 4-bit ripple type decade counter. The device consists of 4-master slave flip flops internally connected to provide a divide by two and divide by 5 sections. Each section has a separate clock i/p to initiate state changes of the counter on the high to low clock transition.

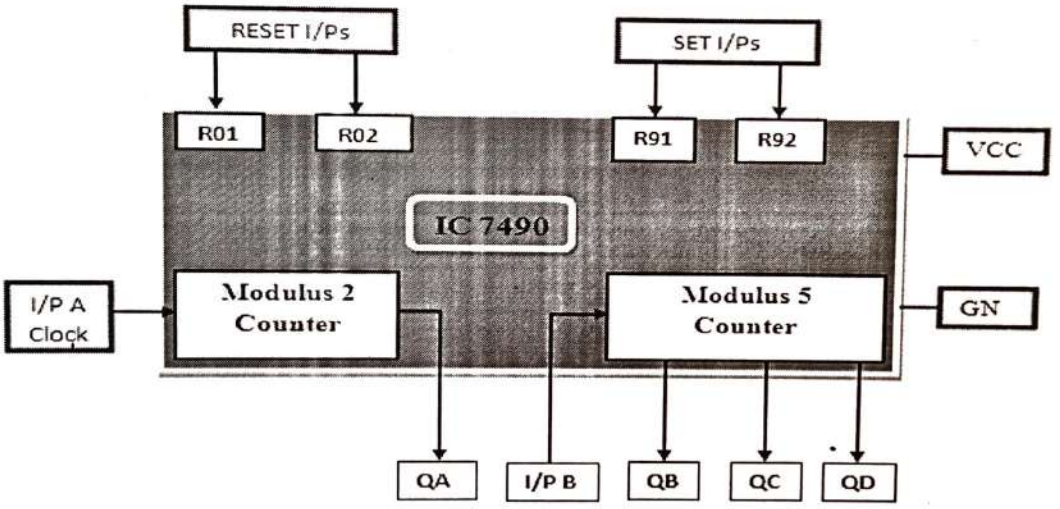
Since the o/p from the divide by 2 section is not internally connected to the succeeding stages. The device may be operated in various counting modes. In a BCD counter the CP_1 input must be externally connected to Q_A o/p. The CP_0 i/p receives the incoming count producing a BCD count sequence. It is also provided with additional gating to provide a divide by 2 counter and binary counter for which the count cycle length is divide by 5. The device may be operated in various counting modes.

There are 2 reset inputs $R_0(1)$ and $R_0(2)$ both of which need to be connected to the 'logic 1' for clearing all flip flops. Two set inputs $R_g(1)$ and $R_g(2)$ when connected to logic 1 are used for setting counter to 1001 (BCD 9).

Pin out of IC 7490:



Basic internal Structure of IC 7490:



Function Table of MOD-5 counter:

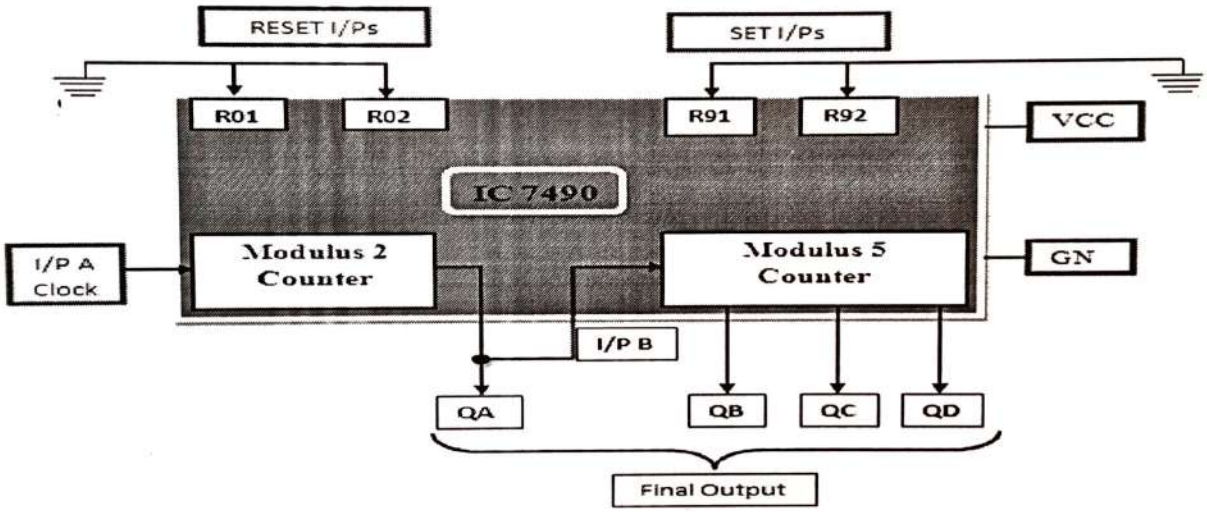
Input B clock	Output			Count
	QD	QC	QB	
	0	0	0	0
	0	0	1	1
	0	1	0	2
	0	1	1	3
	1	0	0	4

Design of MOD-10 counter using IC 7490:

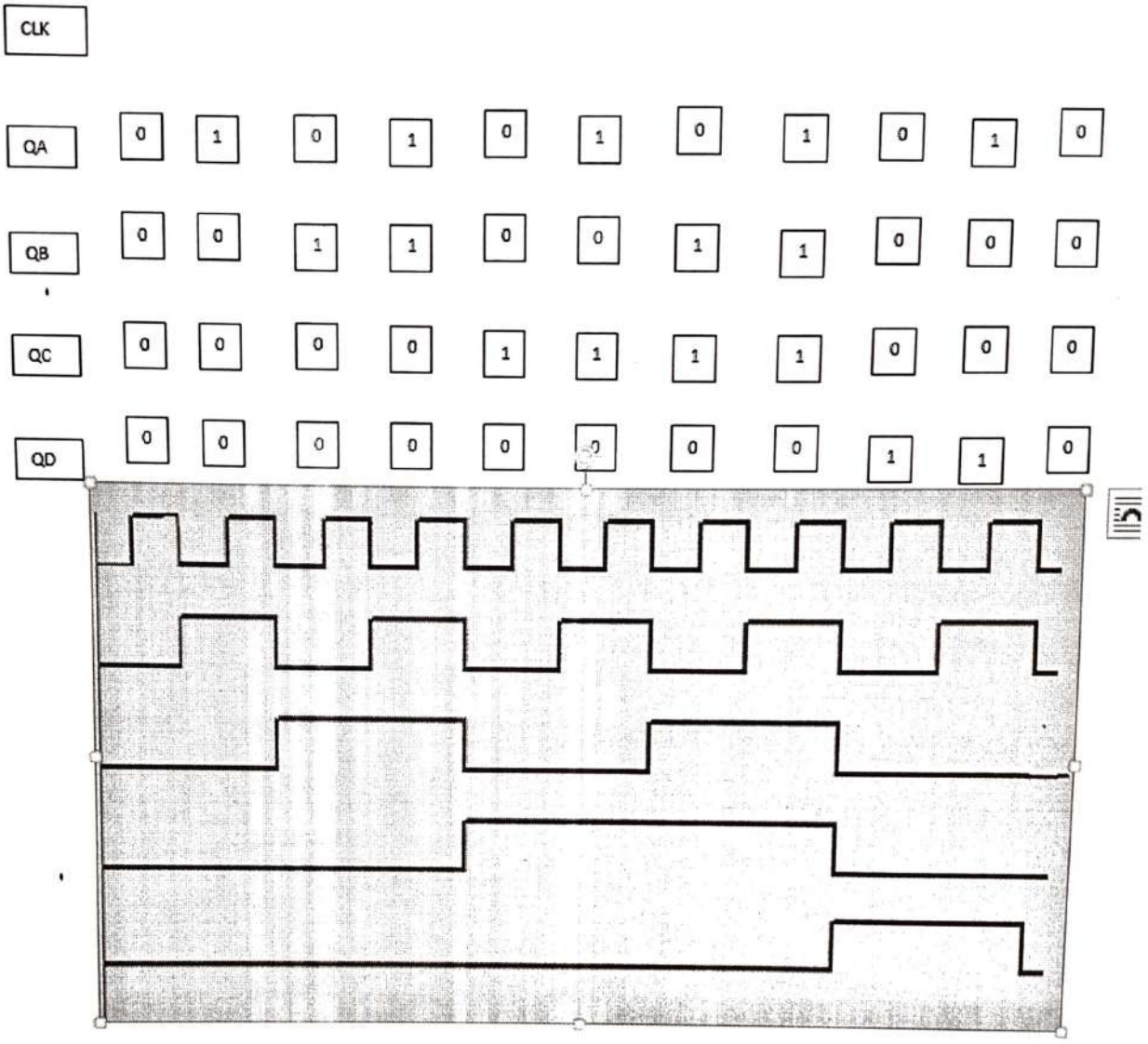
The QA o/p the first flip flop is connected to the input B which is clock i/p of internal MOD-5 ripple counter. Due to cascading of Mod-2 and Mod-5 counters, the overall configuration the decade counters count from 0000 to 1001. After 1001 mod-5 resets to 0000 and next count after 1001 is 0000.

When QA o/p is connected to B i/p, we have the Mod-2 counter followed by Mod-5 counter. The count sequence obtained is shown in the table. It may be noted that QA changes from 0 to 1 the state of Mod-5 counter doesn't change, whereas when QA changes from 1 to 0 the Mod-5 counter goes to the next state.

Logic Diagram MOD-10 counter using IC 7490:



Timing diagram of mod10:



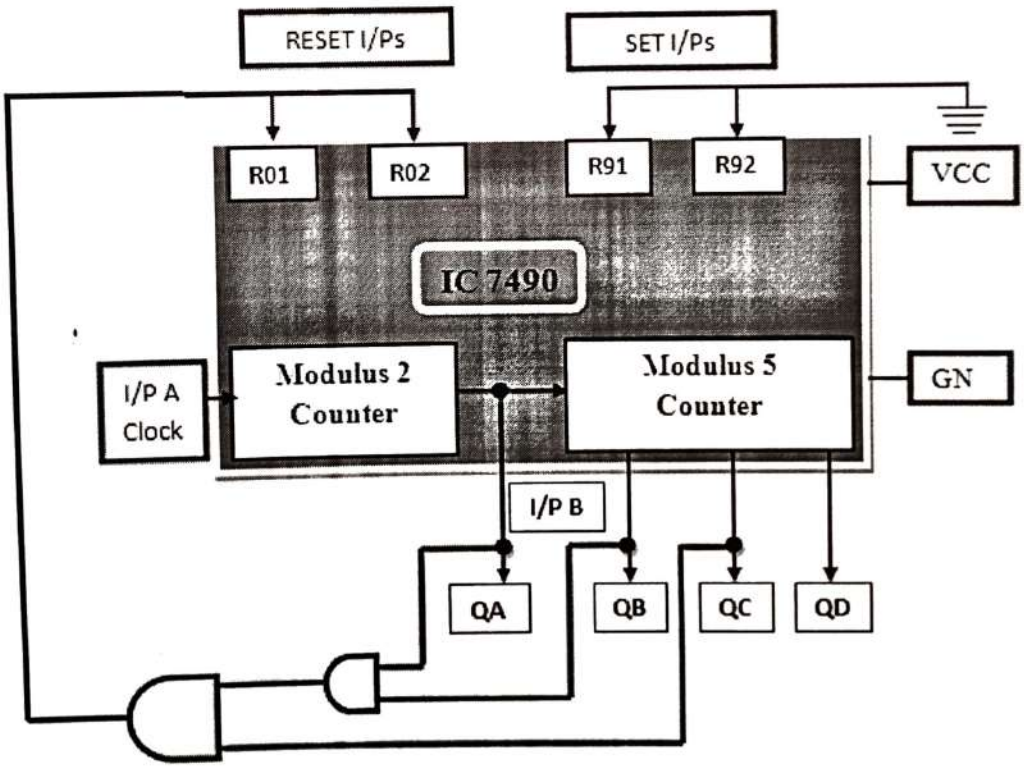
Design of Mod-7 Counter using IC 7490:

Mod-7 counter counts through seven states from 0 to 6 counters and it should reset as soon as the count becomes 7. The o/p of reset logic should be 1 corresponding to invalid states. The reset logic o/p should be applied to pin 2 and 3.

Truth Table of Reset Logic:

QD	QC	QB	QA	Y
0	0	0	0	0
0	0	0	1	0
0	0	1	0	0
0	0	1	1	0
0	1	0	0	0
0	1	0	1	0
0	1	1	0	0
0	1	1	1	1
1	0	0	0	1
1	0	0	1	1

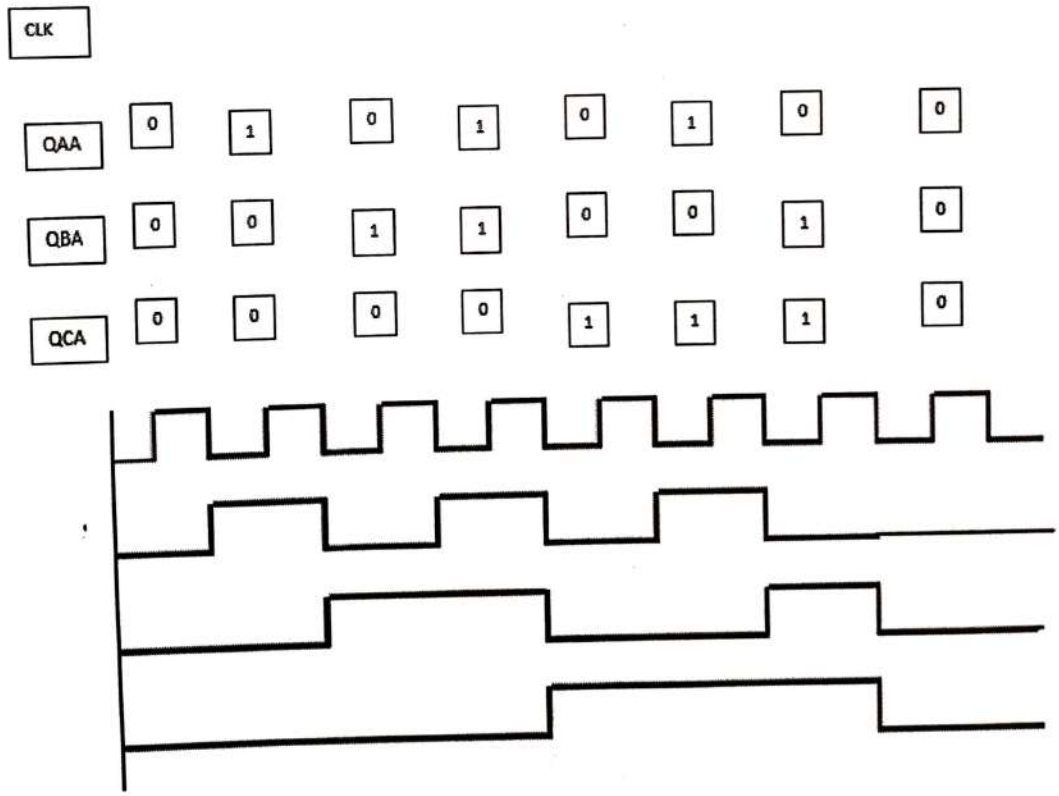
Logic Diagram Mod 7 Counter using IC 7490 :



Function table:

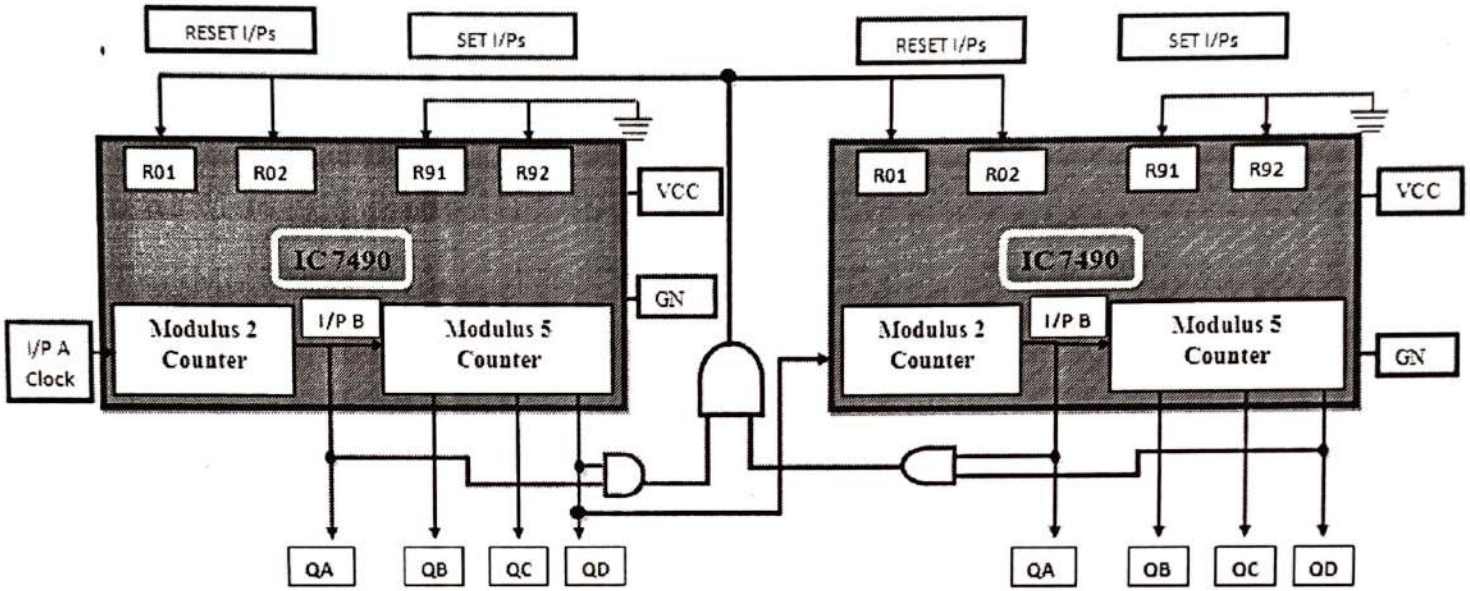
I/p clock	Output				Count
	QD	**QC**	**QB**	**QA**	
	0	**0**	**0**	**0**	**0**
	0	**0**	**0**	**1**	**1**
	0	**0**	**1**	**0**	**2**
	0	**0**	**1**	**1**	**3**
	0	**1**	**0**	**0**	**4**
	0	**1**	**0**	**1**	**5**
	0	**1**	**1**	**0**	**6**

Timing diagram of mod7:



Design of Mod-99 using IC 7490:

For Mod-99 two IC 7490's will be required. Hence to implement a divide by 99 counter we have to use two decade counters IC's. A divide by 99 counter counts 99 states from 0 to 98 and the counter should reset as soon as the count becomes 99. So in order to reset the counter of 99 connect the Q o/p which are equal to 1 in the count of 99 to an 'And' gate & then connect and o/p to the reset i/p of both IC's.

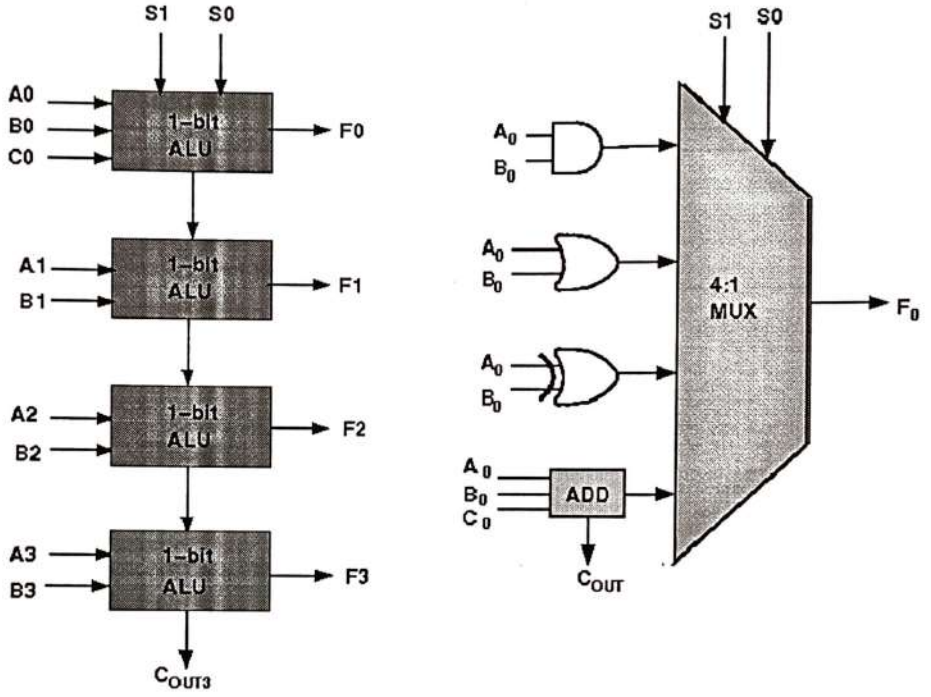


Experiment No. 6:-

Design of Arithmetic Logic Unit:

Theory for **Design of ALU**

ALU or Arithmetic Logical Unit is a digital circuit to do arithmetic operations like addition, subtraction,division, multiplication and logical oparations like and, or, xor, nand, nor etc. A simple block diagram of a 4 bit ALU for operations and,or,xor and Add is shown here :



The 4-bit ALU block is combined using 4 1-bit ALU block

Design Issues :

The circuit functionality of a 1 bit ALU is shown here, depending upon the control signal S_1 and S_0 the circuit operates as follows:

for Control signal $\mathbf{S_1 = 0}$, $S_0 = 0$, the output is **A And B,**

for Control signal $\mathbf{S_1 = 0}$, $S_0 = 1$, the output is **A Or B,**

for Control signal $\mathbf{S_1 = 1}$, $S_0 = 0$, the output is **A Xor B,**

for Control signal $\mathbf{S_1 = 1}$, $S_0 = 1$, the output is **A Add B.**

The truth table for 16-bit ALU with capabilities similar to 74181 is shown here:

Required functionality of ALU (inputs and outputs are active high)

Mode Select Inputs				F_n for active HIGH operands Logic	Arithmetic (note 2)
S3	S2	S1	S0	(M = H)	(M = L) (C_n=L)
L	L	L	L	A'	A
L	L	L	H	A'+B'	A+B
L	L	H	L	A'B	A+B'
L	L	H	H	Logic 0	minus 1
L	H	L	L	(AB)'	A plus AB'
L	H	L	H	B'	(A + B) plus AB'
L	H	H	L	A □ B	A minus B minus 1
L	H	H	H	AB'	AB minus 1
H	L	L	L	A'+B	A plus AB
H	L	L	H	(A □ B)'	A plus B
H	L	H	L	B	(A + B') plus AB
H	L	H	H	AB	AB minus 1
H	H	L	L	Logic 1	A plus A (Note 1)
H	H	L	H	A+B'	(A + B) plus A
H	H	H	L	A+B	(A + B') plus A
H	H	H	H	A	A minus 1

The L denotes the logic low and H denotes logic high.

Objective

Objective of 4 bit arithmetic logic unit (with AND, OR, XOR, ADD operation):

1. Understanding behaviour of arithmetic logic unit from working module and the module designed by the student as part of the experiment
2. Designing an arithmetic logic unit for given parameter

Examining behaviour of arithmetic logic unit for the working module and module designed by the student as part of the experiment (refer to the circuit diagram):

Loading data in the arithmetic logic unit (refer to procedure tab for further detail and experiment manual for pin numbers):

- **load the two input numbers as:**
  - **A(A3 A2 A1 A0): A3=1, A2=1, A1=0, A0=0**
  - **B(B3 B2 B1 B0): B3=1, B2=0, B1=0, B0=1**
  - **carry in(C0)=0**

examining the AND behaviour:

- load data in select input as:
  - S1=0, S0=0 `
- check output:
  - F3=1, F2=0, F1=0, F0=0
  - cout=0 `

examining the OR behaviour:

- load data in select input as:
  - S1=0, S0=1 `
- check output:
  - F3=1, F2=1, F1=0, F0=1
  - cout=0 `

examining the XOR behaviour:

- load data in select input as:
  - S1=1, S0=0 `
- 'check output:
  - F3=0, F2=1, F1=0, F0=1
  - cout=0 `

examining the ADD behaviour:

- load data in select input as:
  - S1=1, S0=1 `
- check output:
  - F3=0, F2=1, F1=0, F0=1
  - cout=1 `

Recommended learning activities for the experiment: Leaning activities are designed in two stages, a basic stage and an advanced stage. Accomplishment of each stage can be self-evaluated through the given set of quiz questions consisting of multiple type and subjective type questions. In the basic stage, it is recommended to perform the experiment firstly, on the given encapsulated working module, secondly, on the module designed by the student, having gone through the theory, objective and procuder. By performing the experiment on the working module, students can only observe the input-output behavior. Where as, performing experiments on the designed module, students can do circuit analysis, error analysis in addition with the input-output behavior. It is recommended to perform the experiments following the given guideline to check behavior and test plans along with their own circuit analysis. Then students are recommended to move on to the advanced stage. The advanced stage includes the accomplishment of the given assignments which will provide deeper understanding of the topic with innovative circuit design experience. At any time, students can mature their knowledge base by further reading the references provided for the experiment.

- color configuration of wire for 5 valued logic supported by the simulator:

- if value is UNKNOWN, wire color= maroon
- if value is TRUE, wire color= blue
- if value is FALSE, wire color= black
- if value is HI IMPEDENCE, wire color= green
- if value is INVALID, wire color= orange

likewise the 16 bit arithmetic logic unit can be designed and tested

- by cascading 4 bit ALUs only the carry will propagate to the next level for ADD operation

Test plan :

1. Set inputs 0101 and 0011 and check output for all possible select input combinations.
2. Set any two 16-bit number and check output for all possible select input combinations.

Use Display units for checking output. Try to use minimum number of components to build. The pin configuration of the canned components are shown when mouse hovered over a component.

Assignment Statements :

1. Design a 4 bit ALU comprising only the AND, OR, XOR and Add operations.
2. Design a 16-bit ALU with capabilities similar to 74181

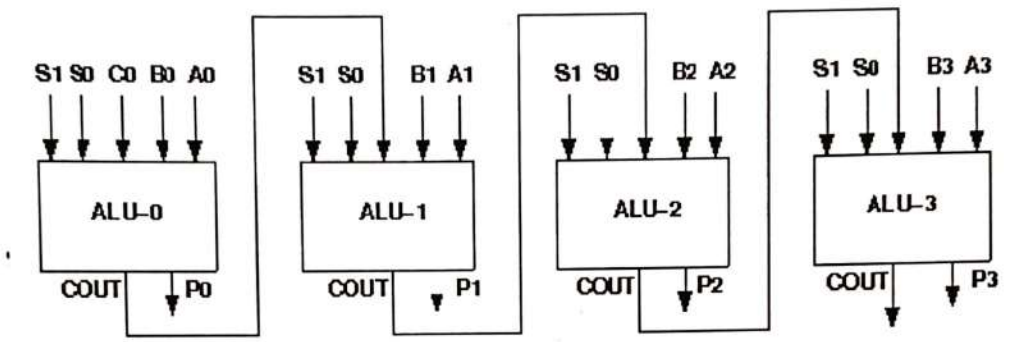
Procedure
Design of ALU:

Procedure to perform the experiment: Design of 4 bit ALU

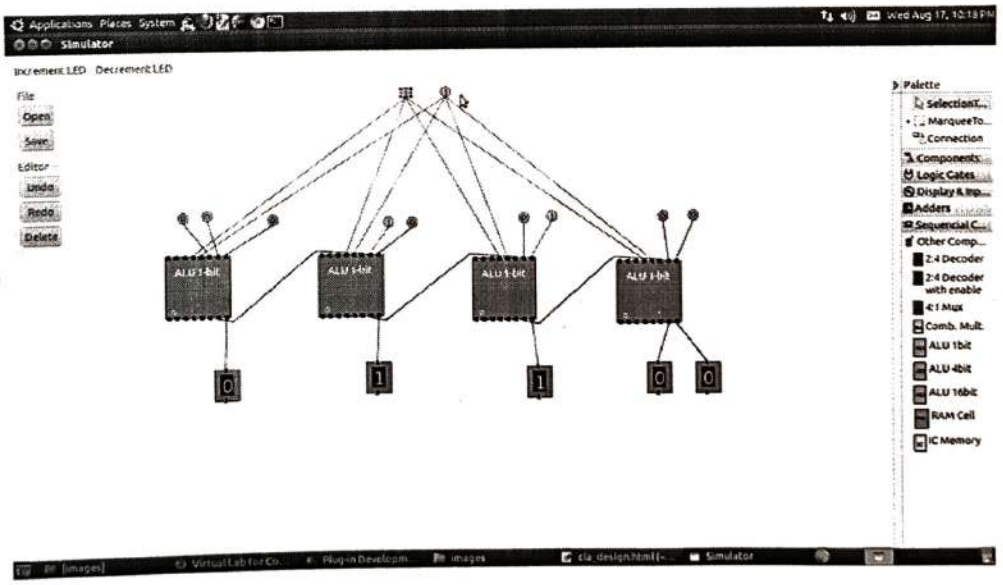
1. Start the simulator as directed. This simulator supports 5-valued logic.
2. To design the circuit we need 4 1-bit ALU, 11 Bit switch (to give input,which will toggle its value with a double click), 5 Bit displays (for seeing output), wires.
3. The pin configuration of a component is shown whenever the mouse is hovered on any canned component of the palette. Pin numbering starts from 1 and from the bottom left corner (indicating with the circle) and increases anticlockwise.
4. For 1-bit ALU input A0 is in pin-9,B0 is in pin-10, C0 is in pin-11 (this is input carry), for selection of operation, S0 is in pin-12, S1 is in pin-13, output F is in pin-8 and output carry is pin-7
5. Click on the 1-bit ALU component (in the Other Component drawer in the pallet) and then click on the position of the editor window where you want to add the component (no drag and drop, simple click will serve the purpose), likewise add 3 more 1-bit ALU (from the Other Component drawer in the pallet), 11 Bit switches and 5 Bit Displays (from Display and Input drawer of the pallet, if it is not seen scroll down in the drawer), 3 digital display and 1 bit Displays (from Display and Input drawer of the pallet, if it is not seen scroll down in the drawer)
6. To connect any two components select the Connection menu of Palette, and then click on the Source terminal and click on the target terminal. According to the circuit diagram connect all the components. Connect the Bit switches with the inputs and Bit displays component with the outputs. After the connection is over click the selection tool in the pallete.

7. See the output, in the screenshot diagram we have given the value of S1 S0=11 which will ,perform add operation and two number input as A0 A1 A2 A3=0010 and B0 B1 B2 B3=0100 so get output F0 F1 F2 F3=0110 as sum and 0 as carry which is indeed an add operation.you can also use many other combination of different values and check the result. The operations are implemented using the truth table for 4 bit ALU given in the theory.

Circuit diagram of 4 bit ALU:



Screenshot of Design of 4 bit ALU:



Components :

To build any 4 bit ALU, we need :

1. AND gate, OR gate, XOR gate

2. Full Adder,
3. 4-to-1 MUX
4. Wires to connect.

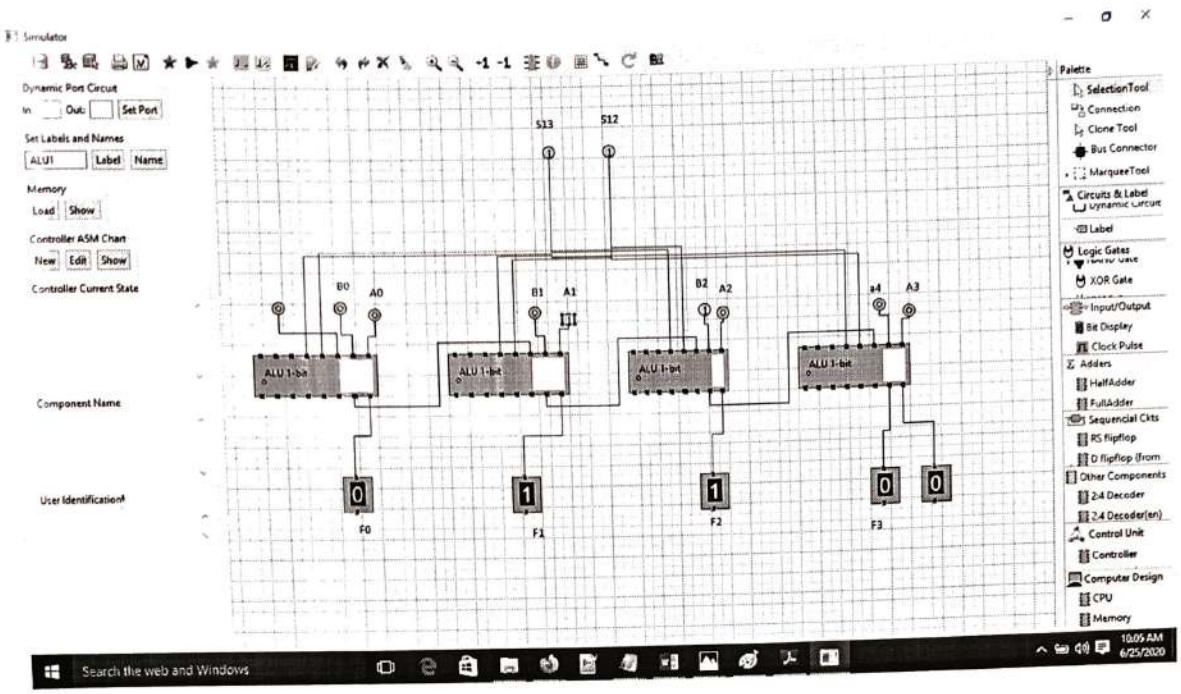
In case of counters the number of flip-flops depends on the number of different states in the counter.

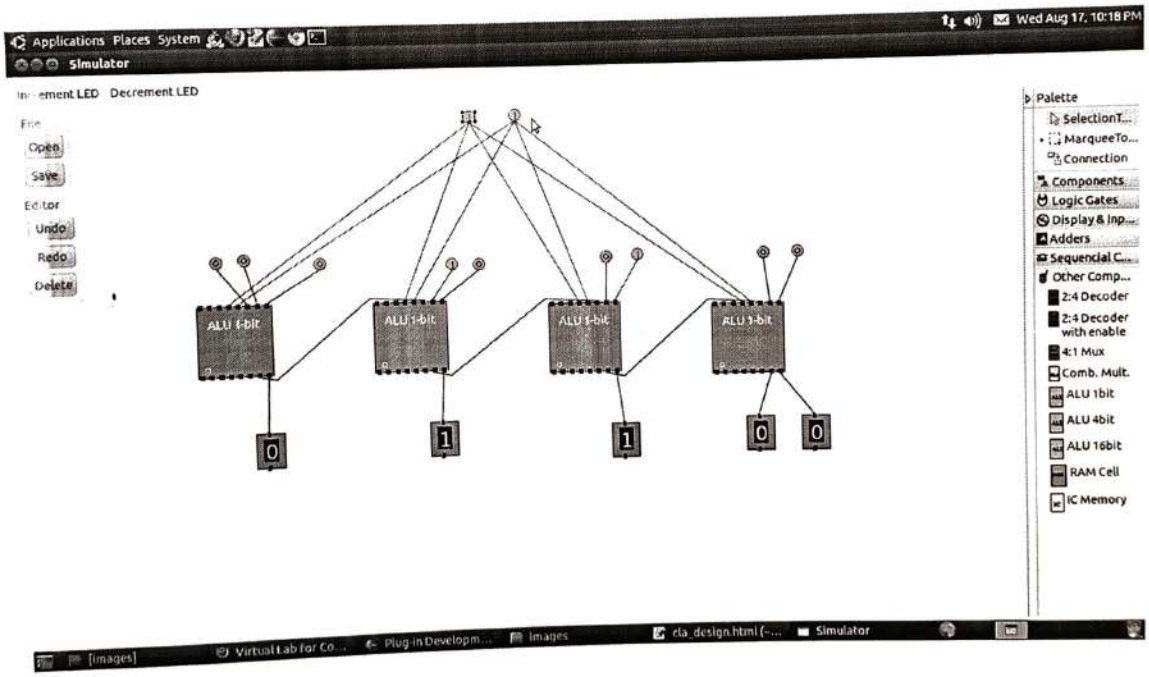
Experiment

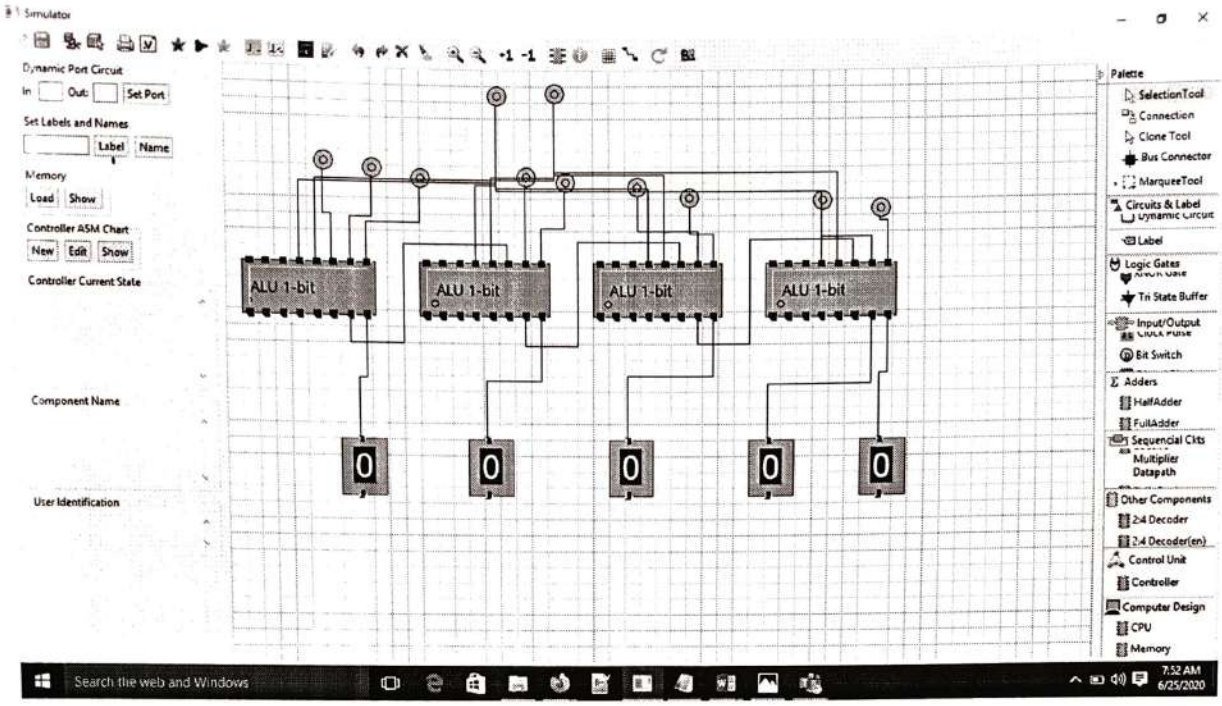
Design of ALU :

General guideline to use the simulator for performing the experiment:

- Start the simulator as directed. For more detail please refer to the manual for using the simulator
- The simulator supports 5-valued logic
- To add the logic components to the editor or canvas (where you build the circuit) select any component and click on the position of the canvas where you want to add the component
- The pin configuration is shown when you select the component and press the 'show pinconfig' button in the left toolbar or whenever the mouse is hovered on any canned component of palette
- To connect any two components select the connection tool of palette, and then click on the source terminal and then click on the the target terminal
- To move any component select the component using the selection tool and drag the component to the desired position
- To give a toggle input to the circuit, use 'Bit Switch' which will toggle its value with a double click
- Use 'Bit Display' component to see any single bit value. 'Digital Display' will show the output in digital format
- undo/redo, delete, zoom in/zoom out, and other functionalities have been given in the top toolbar for ease of circuit building
- Use start/stop clock pulse to start or stop the clock input of the circuit. Clock period can be set from the given 'set clock' button in the left toolbar
- Use 'plot graph' button to see input-output wave forms
- Users can save their circuits with .logic extension and reuse them
- **After building the circuit press the simulate button in the top toolbar to get the output**
- **If the circuit contains a clock pulse input, then the 'start clock' button will start the simulation of the whole circuit. Then there is no need to again press the 'simulate' button.**


Simulator
Dynamic Port Circuit
In Out Set Port
Set Labels and Names
ALU1 Label Name
Memory
Load Show
Controller ASM Chart
New Edit Show
Controller Current State
Component Name
User Identification
S13 S12
B0 A0 B1 A1 B2 A2 a4 A3
ALU 1-bit
F0 F1 F2 F3
Palette
SelectionTool
Connection
Clone Tool
Bus Connector
MarqueeTool
Circuits & Label
Label
Logic Gates
XOR Gate
Input/Output
Bit Display
Clock Pulse
Adders
HalfAdder
FullAdder
Sequencial Ckts
RS flipflop
D flipflop (from
Other Components
2:4 Decoder
2:4 Decoder(en)
Control Unit
Controller
Computer Design
CPU
Memory
Search the web and Windows
10:05 AM
6/25/2020


Applications Places System
Wed Aug 17, 10:18 PM
Simulator
Decrement LED
File
Open
Save
Editor
Undo
Redo
Delete
ALU 1-bit
Palette
SelectionT...
MarqueeTo...
Connection
Components
Logic Gates
Display & Inp...
Adders
Sequencial C...
Other Comp...
2:4 Decoder
2:4 Decoder with enable
4:1 Mux
Comb. Mult.
ALU 1bit
ALU 4bit
ALU 16bit
RAM Cell
IC Memory
[images]
VirtualLab for Co...
Plug-in Developm...
Images
cla_design.html (~...
Simulator


Simulator
Dynamic Port Circuit
In
Out
Set Port
Set Labels and Names
Label
Name
Memory
Load
Show
Controller ASM Chart
New
Edit
Show
Controller Current State
Component Name
User Identification
ALU 1-bit
ALU 1-bit
ALU 1-bit
ALU 1-bit
0
0
0
0
0
Palette
SelectionTool
Connection
Clone Tool
Bus Connector
MarqueeTool
Circuits & Label
Label
Logic Gates
Tri State Buffer
Input/Output
Bit Switch
Adders
HalfAdder
FullAdder
Sequencial Ckts
Multiplier
Datapath
Other Components
2:4 Decoder
2:4 Decoder(en)
Control Unit
Controller
Computer Design
CPU
Memory
Search the web and Windows
7:52 AM
6/25/2020